Data Pre-Processing

Dataset:

Imports and data loading:

```
import os
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import skew
from dython.nominal import cramers_v, correlation_ratio

# Load the dataset

filename = "compas-scores-two-years.csv"
path = os.path.join(os.getcwd(), '..', 'data', filename)
df = pd.read_csv(path)
```

Dataset Overview:

```
print(f"Dataset Shape: {df.shape}")
print(f"Count of Unique Individuals: {df['id'].nunique()}")
```

```
Dataset Shape: (7214, 53)
Count of Unique Individuals: 7214
```

Variable Types

```
print("\nData types:")
print(df.dtypes)
```

```
Data types:
id                           int64
name                           str
first                          str
last                           str
compas_screening_date          str
sex                            str
dob                            str
age                          int64
age_cat                        str
race                           str
juv_fel_count                int64
decile_score                 int64
juv_misd_count               int64
juv_other_count              int64
priors_count                 int64
days_b_screening_arrest    float64
c_jail_in                      str
c_jail_out                     str
c_case_number                  str
c_offense_date                 str
c_arrest_date                  str
c_days_from_compas         float64
c_charge_degree                str
c_charge_desc                  str
is_recid                     int64
r_case_number                  str
r_charge_degree                str
r_days_from_arrest         float64
r_offense_date                 str
r_charge_desc                  str
r_jail_in                      str
r_jail_out                     str
violent_recid              float64
is_violent_recid             int64
vr_case_number                 str
vr_charge_degree               str
vr_offense_date                str
vr_charge_desc                 str
type_of_assessment             str
decile_score.1               int64
score_text                     str
screening_date                 str
v_type_of_assessment           str
v_decile_score               int64
v_score_text                   str
v_screening_date               str
in_custody                     str
out_custody                    str
priors_count.1               int64
start                        int64
end                          int64
event                        int64
two_year_recid               int64
dtype: object
```

Null Values

```
print("\nNull or NAN counts:")
print(df.isnull().sum())
```

```
Null or NAN counts:
id                            0
name                          0
first                         0
last                          0
compas_screening_date         0
sex                           0
dob                           0
age                           0
age_cat                       0
race                          0
juv_fel_count                 0
decile_score                  0
juv_misd_count                0
juv_other_count               0
priors_count                  0
days_b_screening_arrest     307
c_jail_in                   307
c_jail_out                  307
c_case_number                22
c_offense_date             1159
c_arrest_date              6077
c_days_from_compas           22
c_charge_degree               0
c_charge_desc                29
is_recid                      0
r_case_number              3743
r_charge_degree            3743
r_days_from_arrest         4898
r_offense_date             3743
r_charge_desc              3801
r_jail_in                  4898
r_jail_out                 4898
violent_recid              7214
is_violent_recid              0
vr_case_number             6395
vr_charge_degree           6395
vr_offense_date            6395
vr_charge_desc             6395
type_of_assessment            0
decile_score.1                0
score_text                    0
screening_date                0
v_type_of_assessment          0
v_decile_score                0
v_score_text                  0
v_screening_date              0
in_custody                  236
out_custody                 236
priors_count.1                0
start                         0
end                           0
event                         0
two_year_recid                0
dtype: int64
```

Numerical Variable Analysis

```
df.describe()
```

	id	age	juv_fel_count	decile_score	juv_misd_count	juv_other_count	priors_count	days_b_screening_arrest	c_days_from_compas	is_recid	r_days_from_arrest	violent_recid	is_violent_recid
count	7214.000000	7214.000000	7214.000000	7214.000000	7214.000000	7214.000000	7214.000000	6907.000000	7192.000000	7214.000000	2316.000000	0.0	7214.000000
mean	5501.255753	34.817993	0.067230	4.509565	0.090934	0.109371	3.472415	3.304763	57.731368	0.481148	20.269430	NaN	0.113529
std	3175.706870	11.888922	0.473972	2.856396	0.485239	0.501586	4.882538	75.809505	329.740215	0.499679	74.871668	NaN	0.317261
min	1.000000	18.000000	0.000000	1.000000	0.000000	0.000000	0.000000	-414.000000	0.000000	0.000000	-1.000000	NaN	0.000000
25%	2735.250000	25.000000	0.000000	2.000000	0.000000	0.000000	0.000000	-1.000000	1.000000	0.000000	0.000000	NaN	0.000000
50%	5509.500000	31.000000	0.000000	4.000000	0.000000	0.000000	2.000000	-1.000000	1.000000	0.000000	0.000000	NaN	0.000000
75%	8246.500000	42.000000	0.000000	7.000000	0.000000	0.000000	5.000000	0.000000	2.000000	1.000000	1.000000	NaN	0.000000
max	11001.000000	96.000000	20.000000	10.000000	13.000000	17.000000	38.000000	1057.000000	9485.000000	1.000000	993.000000	NaN	1.000000

Categorical Variable Analysis

```
exclude_vars = [
    # Identifiers & High-Cardinality Text
    'name', 'first', 'last', 'c_case_number', 'r_case_number', 'vr_case_number',
    # 'c_charge_desc', 'r_charge_desc', 'vr_charge_desc',

    # Timestamps & Dates
    'compas_screening_date', 'screening_date', 'v_screening_date', 'dob',
    'c_jail_in', 'c_jail_out', 'c_offense_date', 'c_arrest_date',
    'r_offense_date', 'r_jail_in', 'r_jail_out', 'vr_offense_date',
    'in_custody', 'out_custody',

    # Redundant Columns
    # 'age_cat', 'score_text', 'v_score_text'
]
categorical_vars = [col for col in df.select_dtypes(include=['object', 'category', 'string']).columns if col not in exclude_vars]
for col in categorical_vars:
    print(f"Col: {col}")
    print(f"Number of categories: {df[col].nunique()}")
    print("Percentage:")
    print(df[col].value_counts(normalize=True).mul(100).round(2))
    print()
```

```
Col: sex
Number of categories: 2
Percentage:
sex
Male      80.66
Female    19.34
Name: proportion, dtype: float64

Col: age_cat
Number of categories: 3
Percentage:
age_cat
25 - 45            56.96
Greater than 45    21.85
Less than 25       21.19
Name: proportion, dtype: float64

Col: race
Number of categories: 6
Percentage:
race
African-American    51.23
Caucasian           34.02
Hispanic             8.83
Other                5.23
Asian                0.44
Native American      0.25
Name: proportion, dtype: float64

Col: c_charge_degree
Number of categories: 2
Percentage:
c_charge_degree
F    64.68
M    35.32
Name: proportion, dtype: float64

Col: c_charge_desc
Number of categories: 437
Percentage:
c_charge_desc
Battery                          16.09
arrest case no charge            15.82
Possession of Cocaine             6.60
Grand Theft in the 3rd Degree     5.92
Driving While License Revoked     2.78
                                  ...
Purchase Of Cocaine               0.01
Deliver Cocaine 1000FT Park       0.01
Burglary Structure Occupied       0.01
Attempted Deliv Control Subst     0.01
Possession of XLR11               0.01
Name: proportion, Length: 437, dtype: float64

Col: r_charge_degree
Number of categories: 10
Percentage:
r_charge_degree
(M1)     34.60
(M2)     31.89
(F3)     25.70
(F2)      4.84
(F1)      1.47
(MO3)     1.12
(F7)      0.20
(F6)      0.09
(CO3)     0.06
(F5)      0.03
Name: proportion, dtype: float64

Col: r_charge_desc
Number of categories: 340
Percentage:
r_charge_desc
Driving License Suspended           7.56
Possess Cannabis/20 Grams Or Less   7.41
Resist/Obstruct W/O Violence        5.89
Battery                             5.63
Operating W/O Valid License         5.04
                                    ...
Murder in 2nd Degree                0.03
Food License Violation              0.03
Failure To Return Hired Vehicle     0.03
Defrauding Innkeeper $300/Less      0.03
Petit Theft Habitual Offender       0.03
Name: proportion, Length: 340, dtype: float64

Col: vr_charge_degree
Number of categories: 9
Percentage:
vr_charge_degree
(M1)     42.00
(F3)     27.84
(F2)     19.78
(F1)      4.64
(M2)      2.32
(F7)      2.20
(MO3)     0.61
(F6)      0.49
(F5)      0.12
Name: proportion, dtype: float64

Col: vr_charge_desc
Number of categories: 83
Percentage:
vr_charge_desc
Battery                          40.17
Felony Battery (Dom Strang)       4.64
Battery on Law Enforc Officer     4.64
Aggravated Assault W/Dead Weap    4.52
Aggrav Battery w/Deadly Weapon    4.15
                                  ...
Shoot In Occupied Building        0.12
Vehicular Homicide                0.12
Murder in 2nd Degree              0.12
Arson II (Vehicle)                0.12
Att Robbery Sudd Snatch No Weap   0.12
Name: proportion, Length: 83, dtype: float64

Col: type_of_assessment
Number of categories: 1
Percentage:
type_of_assessment
Risk of Recidivism    100.0
Name: proportion, dtype: float64

Col: score_text
Number of categories: 3
Percentage:
score_text
Low       54.02
Medium    26.53
High      19.45
Name: proportion, dtype: float64

Col: v_type_of_assessment
Number of categories: 1
Percentage:
v_type_of_assessment
Risk of Violence    100.0
Name: proportion, dtype: float64

Col: v_score_text
Number of categories: 3
Percentage:
v_score_text
Low       66.00
Medium    24.11
High       9.90
Name: proportion, dtype: float64
```

Data Cleaning

Duplicate Columns

There are 2 columns with duplicates in the dataset: `decile_score` and `priors_count`. The `screening_date` seems to also be a duplicate of the `compas_screening_date`.

We'll confirm that all are truly duplicate before taking any decisions.

```
print(f"decile_score and decile_score.1 are identical: {df['decile_score'].equals(df['decile_score.1'])}")
print(f"priors_count and priors_count.1 are identical: {df['priors_count'].equals(df['priors_count.1'])}")
print(f"screening_date and compas_screening_date are identical: {df['screening_date'].equals(df['compas_screening_date'])}")
```

```
decile_score and decile_score.1 are identical: True
priors_count and priors_count.1 are identical: True
screening_date and compas_screening_date are identical: True
```

Since all are truly duplicates, we'll remove `decile_score.1` and `priors_count.1` and `compas_screening_date`.

```
df = df.drop(columns=['decile_score.1', 'priors_count.1', 'compas_screening_date'])
```

0 Variance Columns

There are 2 columns with 0 variance in the dataset: `type_of_assessment` and `v_type_of_assessment`.

We'll remove both of them, since they are not providing any information for the model.

```
print(df["type_of_assessment"].value_counts(normalize=True).mul(100).round(2), end="\n\n")
print(df["v_type_of_assessment"].value_counts(normalize=True).mul(100).round(2))

df = df.drop(columns=['type_of_assessment', 'v_type_of_assessment'])
```

```
type_of_assessment
Risk of Recidivism    100.0
Name: proportion, dtype: float64

v_type_of_assessment
Risk of Violence    100.0
Name: proportion, dtype: float64
```

Irrelevant Columns

There are a couple of columns that don't provide any relevant information for the models and just act as noise.

These are mostly text columns that are not neccesarily categorical:

- `id`
- `name`, `first`, `last`
- `c_case_number`, `r_case_number`, `vr_case_number`

```
columns_to_drop = [
    'id', 'name', 'first', 'last',
    'c_case_number', 'r_case_number', 'vr_case_number'
]

df = df.drop(columns=columns_to_drop)
```

Redundant Columns

These are columns that are redundant, because there are already other columns with equivalent information more fit for more model training: (the columns after the arrow are the ones that best represent the information)

- `age_cat` -> `age`
- `score_text`, `v_score_text` -> `decile_score`, `v_decile_score`
- `c_charge_desc`, `r_charge_desc`, `vr_charge_desc` -> `c_charge_degree`, `r_charge_degree`, `vr_charge_degree`

Just to be sure, we calculated the correlations between these fields.

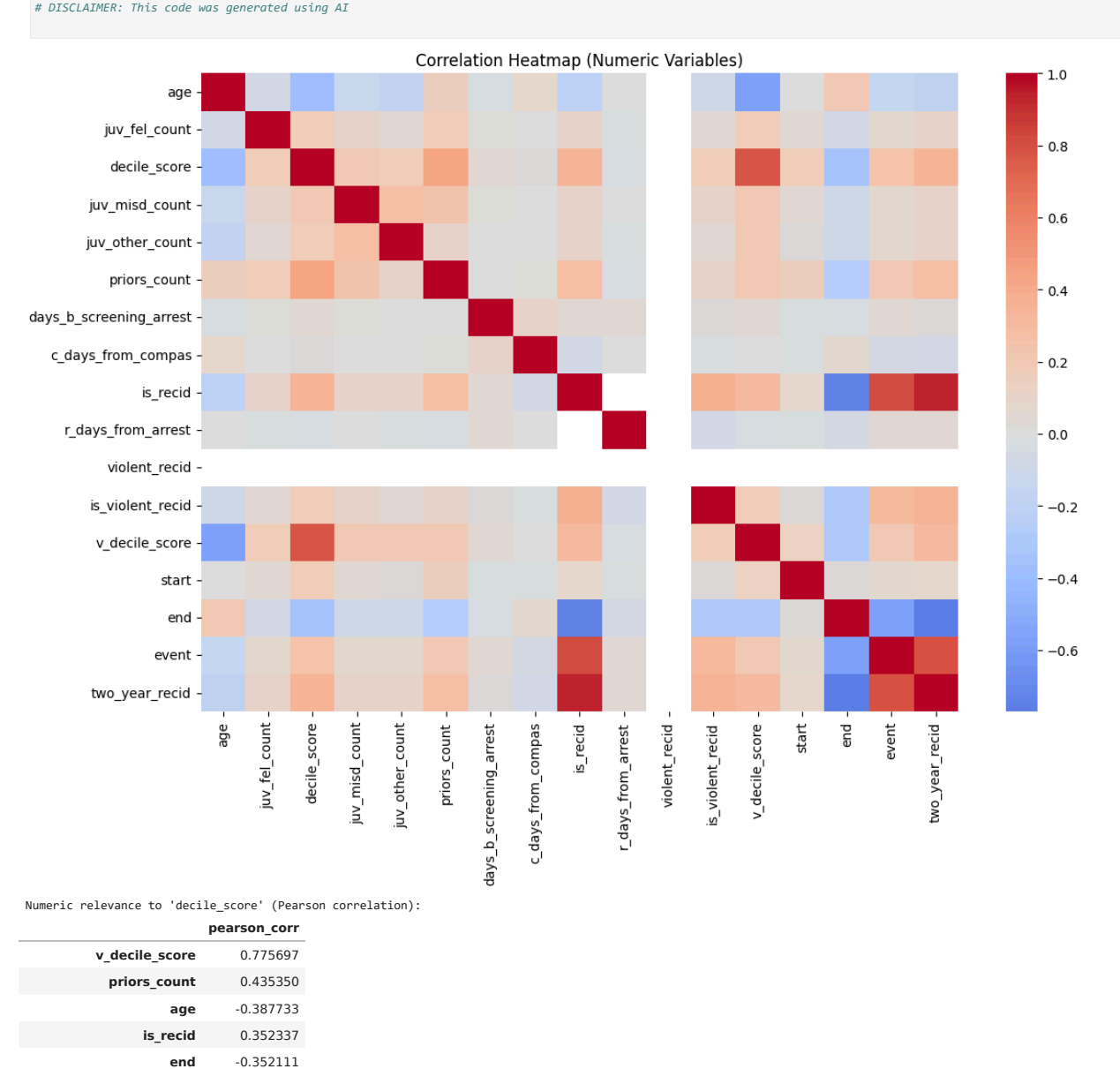
```
results = []

# Define your pairs explicitly: (Column to potentially drop, Column to test against, Method)
study_configs = [
    ("age_cat", "age", "eta"),
    ("score_text", "decile_score", "eta"),
    ("v_score_text", "v_decile_score", "eta"),
    # ("c_charge_desc", "c_charge_degree", "cramers_v"),
    ("r_charge_desc", "r_charge_degree", "cramers_v"),
    ("vr_charge_desc", "vr_charge_degree", "cramers_v"),
]

for target_col, reference_col, method in study_configs:
    # Ensure columns exist in df before testing
    if target_col in df.columns and reference_col in df.columns:
        if method == "eta":
            val = correlation_ratio(df[target_col], df[reference_col])
        else:
            val = cramers_v(df[target_col], df[reference_col])

        results.append({
            "target_col": target_col,
            "reference_col": reference_col,
            "metric": method,
            "value": val
        })

# Create study DataFrame
corr_study = pd.DataFrame(results)
display(corr_study)

# 1. Start with your base identification columns

# 2. Add columns from study where correlation is >= 0.85 (rounded)
# We round to 2 decimal places to catch 0.846+
redundant_cols = corr_study[corr_study['value'].round(2) >= 0.85]['target_col'].tolist()

print(f"Final columns identified for removal: {redundant_cols}")

# 3. Perform the drop
df = df.drop(columns=[c for c in redundant_cols if c in df.columns])

# DISCLAIMER: This code was generated using AI
```

	target_col	reference_col	metric	value
0	age_cat	age	eta	0.898127
1	score_text	decile_score	eta	0.936545
2	v_score_text	v_decile_score	eta	0.914027
3	r_charge_desc	r_charge_degree	cramers_v	0.951749
4	vr_charge_desc	vr_charge_degree	cramers_v	0.964059

```
Final columns identified for removal: ['age_cat', 'score_text', 'v_score_text', 'r_charge_desc', 'vr_charge_desc']
```

Studying Variable correlations to target variable

```
from scipy.stats import chi2_contingency

# Correlation (numeric) & relevance to target (numeric and categorical)
target = "decile_score"

# ---------- Numeric correlation ----------
numeric_cols = df.select_dtypes(include="number").columns.tolist()
corr_matrix = df[numeric_cols].corr()

# print("Numeric Correlation Matrix:")
# display(corr_matrix)


plt.figure(figsize=(12, 9))
sns.heatmap(corr_matrix, cmap="coolwarm", center=0, square=False)
plt.title("Correlation Heatmap (Numeric Variables)")
plt.tight_layout()
plt.show()

if target in corr_matrix.columns:
    corr_with_target = (
        corr_matrix[target]
        .drop(labels=[target], errors="ignore")
        .sort_values(key=lambda s: s.abs(), ascending=False)
    )
    print(f"\nNumeric relevance to '{target}' (Pearson correlation):")
    display(corr_with_target.to_frame(name="pearson_corr"))


# ---------- Categorical relevance (Cramér's V) ----------

def cramers_v(x, y):
    table = pd.crosstab(x, y)
    if table.empty:
        return float("nan")
    chi2 = chi2_contingency(table)[0]
    n = table.to_numpy().sum()
    if n <= 1:
        return float("nan")
    r, k = table.shape
    phi2 = chi2 / n
    phi2_corr = max(0, phi2 - ((k - 1) * (r - 1)) / (n - 1))
    r_corr = r - ((r - 1) ** 2) / (n - 1)
    k_corr = k - ((k - 1) ** 2) / (n - 1)
    denom = min(k_corr - 1, r_corr - 1)
    return (phi2_corr / denom) ** 0.5 if denom > 0 else float("nan")

if target in df.columns:
    cat_cols = df.select_dtypes(include=["object", "category", "string"]).columns.tolist()
    cat_relevance = pd.Series(
        {c: cramers_v(df[c], df[target]) for c in cat_cols if c != target},
        name="cramers_v"
    ).sort_values(ascending=False)

    print(f"\nCategorical relevance to '{target}' (Cramér's V):")
    display(cat_relevance.to_frame())

# DISCLAIMER: This code was generated using AI
```



```
Numeric relevance to 'decile_score' (Pearson correlation):
```

	pearson_corr
v_decile_score	0.775697
priors_count	0.435350
age	-0.387733
is_recid	0.352337
end	-0.352111
two_year_recid	0.350703
event	0.246873
juv_misd_count	0.215927
juv_other_count	0.183463
juv_fel_count	0.177758
is_violent_recid	0.171292
start	0.170209
days_b_screening_arrest	0.045505
r_days_from_arrest	-0.036378
c_days_from_compas	0.030330
violent_recid	NaN

```
Categorical relevance to 'decile_score' (Cramér's V):
```

	cramers_v
c_charge_degree	0.191363
race	0.145843
c_charge_desc	0.121992
out_custody	0.120809
c_arrest_date	0.114041
c_offense_date	0.106020
in_custody	0.100623
r_jail_out	0.090674
screening_date	0.083142
v_screening_date	0.083142
sex	0.073693
r_charge_degree	0.057152
vr_offense_date	0.045184
vr_charge_degree	0.040874
dob	0.000000
c_jail_out	0.000000
r_offense_date	0.000000
r_jail_in	0.000000
c_jail_in	NaN

Future variables

There's a bunch of variables in the dataset that the model should not have access to.

This is because these features were obtained after the COMPAS score was calculated.

For that reason, I'm going to remove all those features, to avoid data leakage.

```
columns_to_drop = [
    'c_jail_in',
    'c_jail_out',
    'is_recid',
    'r_charge_degree',
    'r_days_from_arrest',
    'r_offense_date',
    'r_jail_in',
    'r_jail_out',
    'violent_recid',
    'is_violent_recid',
    'vr_charge_degree',
    'vr_offense_date',
    'v_decile_score',
    'v_screening_date',
    'in_custody',
    'out_custody',
    'start',
    'end',
    'event',
    'two_year_recid',
    'in_custody',
    'out_custody'
]

df = df.drop(columns=columns_to_drop)

df.columns.tolist()
```

```
['sex',
 'dob',
 'age',
 'race',
 'juv_fel_count',
 'decile_score',
 'juv_misd_count',
 'juv_other_count',
 'priors_count',
 'days_b_screening_arrest',
 'c_offense_date',
 'c_arrest_date',
 'c_days_from_compas',
 'c_charge_degree',
 'c_charge_desc',
 'screening_date']
```

Date Validations

There's not a single row where c_offense_date and c_arrest_date are both not null. I'll take both the columns and merge them into one.

```
df['dob'] = pd.to_datetime(df['dob'], errors='coerce')


both_not_null = df[df['c_offense_date'].notna() & df['c_arrest_date'].notna()]
print(f"Total rows with both dates: {len(both_not_null)}")

df['event_date'] = df['c_offense_date'].combine_first(df['c_arrest_date'])
print(f"Total rows with null event_date: {df['event_date'].isna().sum() / len(df) :.2%}")

df= df.dropna(subset=['event_date'])
df['event_date'] = pd.to_datetime(df['event_date'], errors='coerce')

df['event_year'] = df['event_date'].dt.year

print(f"Total rows with null event_date: {df['event_date'].isna().sum() / len(df) :.2%}")

df = df.drop(columns=['c_offense_date', 'c_arrest_date'])
```

```
Total rows with both dates: 0
Total rows with null event_date: 0.30%
Total rows with null event_date: 0.00%
```

We must now filter the rows where the offense or arrest took effect too far in the timeline from the compas screening.

```
valid_timeline = (df['days_b_screening_arrest'] <= 30) & (df['days_b_screening_arrest'] >= -30)
df = df[valid_timeline]

columns_to_drop = [
    'days_b_screening_arrest',
    'c_days_from_compas',
    'screening_date',
]

df = df.drop(columns=columns_to_drop)
```

Race Misrepresentation

There are some races that severly under-represented in this dataset.

For that reason, we will group every race under 1% of representation into the 'other' race category.

```
# 1. Set your threshold (1% = 0.01)
threshold = 0.01

# 2. Calculate the percentage of each race in the dataset
race_percentages = df['race'].value_counts(normalize=True)

# 3. Create a list of the races that fall strictly below the threshold
races_to_group = race_percentages[race_percentages < threshold].index
print(f"Races being grouped into 'Other': {list(races_to_group)}")

# 4. Replace those specific values with 'Other'
df['race'] = df['race'].replace(races_to_group, 'Other')

df['race'] = df['race'].str.lower().str.strip()

# Verify the new distribution
print("\nNew Race Distribution (%):")
print(df['race'].value_counts(normalize=True).mul(100).round(2))
```

```
Races being grouped into 'Other': ['Asian', 'Native American']

New Race Distribution (%):
race
african-american    51.44
caucasian           34.07
hispanic             8.25
other                6.24
Name: proportion, dtype: float64
```

Feature Engineering

Im going to try and extract some features from the current dataset, because it was reduced by quite a bit.

```
df['age_at_event'] = (df['event_date'] - pd.to_datetime(df['dob'], errors='coerce')).dt.days // 365.25

df['adult_years'] = df['age_at_event'] - 18
df['adult_years'] = df['adult_years'].clip(lower=1)

df['priors_per_adult_year'] = df['priors_count'] / df['adult_years']

# Magnitude: Total Juvenile Offenses
df['total_juv_count'] = df['juv_fel_count'] + df['juv_misd_count'] + df['juv_other_count']

# Binary: Does a juvenile record exist? (1 = Yes, 0 = No)
df['has_juv_record'] = (df['total_juv_count'] > 0).astype(int)

df['first_offense'] = ((df['priors_count'] == 0) & ~df['has_juv_record']).astype(int)

def categorize_charge(description):
    desc = str(description).lower()
    if any(word in desc for word in ['assault', 'battery', 'murder', 'manslaughter', 'robbery']):
        return 'violent'
    elif any(word in desc for word in ['drug', 'cocaine', 'marijuana', 'possession']):
        return 'drug'
    elif any(word in desc for word in ['theft', 'burglary', 'fraud', 'trespass']):
        return 'property'
    elif any(word in desc for word in ['driving', 'license', 'traffic', 'dui']):
        return 'driving'
    else:
        return 'other'

df['charge_category'] = df['c_charge_desc'].apply(categorize_charge)

df['young_man'] = ((df['age_at_event'] < 25) & (df['sex'] == 'Male')).astype(int)

df['velocity_repeat_offender'] = ((df['priors_per_adult_year'] > 1.0) & (df['charge_category'] == 'violent') & (df['c_charge_degree'] == 'F')).astype(int)

df = df.drop(['c_charge_desc', 'adult_years', 'event_date', 'dob'], axis=1)
# This code was generated mostly using AI
```

- `age_at_event`: Age at which the subject commited the offense / was arrested.
- `priors_per_adult_year`: Ratio of adult crimes comitted by the amount of adult years of the subject.
- `total_juv_count`: Total amount of juvenile charges.
- `has_juv_record`: Boolean for if a subject has a juvenile record.
- `first_offense`: Boolean for if a subject has any criminal record.
- `charge_category`: Normalized value for `c_charge_desc`.
- `young_man`: Flag for when a subject is a man less than 25 years old.
- `velocity_repeat_offender`: Flag for when a subject has one or more offenses per year and the charge is a felony associated with violence. Essentially a flag for dangerous repeat offenders.

Also created boolean flags for categorical values.

```
categorical_cols = ['sex', 'race', 'c_charge_degree', 'charge_category']
for col in categorical_cols:
    df[col] = df[col].str.lower().str.strip()

df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
# Convert all boolean columns to 1s and 0s
bool_cols = df.select_dtypes(include='bool').columns
df[bool_cols] = df[bool_cols].astype(int)
```

Final Dataset

```
df.shape
```

```
(6172, 23)
```

```
df.dtypes
```

```
age                           int64
juv_fel_count                 int64
decile_score                  int64
juv_misd_count                int64
juv_other_count               int64
priors_count                  int64
event_year                    int32
age_at_event                float64
priors_per_adult_year       float64
total_juv_count               int64
has_juv_record                int64
first_offense                 int64
young_man                     int64
velocity_repeat_offender      int64
sex_male                      int64
race_caucasian                int64
race_hispanic                 int64
race_other                    int64
c_charge_degree_m             int64
charge_category_drug          int64
charge_category_other         int64
charge_category_property      int64
charge_category_violent       int64
dtype: object
```

```
df.isna().sum()
```

```
age                         0
juv_fel_count               0
decile_score                0
juv_misd_count              0
juv_other_count             0
priors_count                0
event_year                  0
age_at_event                0
priors_per_adult_year       0
total_juv_count             0
has_juv_record              0
first_offense               0
young_man                   0
velocity_repeat_offender    0
sex_male                    0
race_caucasian              0
race_hispanic               0
race_other                  0
c_charge_degree_m           0
charge_category_drug        0
charge_category_other       0
charge_category_property    0
charge_category_violent     0
dtype: int64
```

```
df.to_csv(os.path.join(os.getcwd(), '..', 'data', 'compas_preprocessed.csv'), index=False)
```

0) Enviroment

Imports and Constants

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib, os

from sklearn.model_selection import train_test_split, GridSearchCV, cross_validate, learning_curve
from sklearn.preprocessing import PolynomialFeatures, SplineTransformer, StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.metrics import precision_score, recall_score, f1_score, classification_report, confusion_matrix

from sklearn.linear_model import ElasticNet, QuantileRegressor
from sklearn.ensemble import RandomForestRegressor

from sklearn.pipeline import make_pipeline
from sklearn.compose import TransformedTargetRegressor
from mord import LogisticAT


DATA_PATH = "../data/compas_preprocessed.csv"
_RS = 42
_TARGET = "decile_score"
_TEST_PERCENTAGE = 0.2
```

Import data and isolate target

```
df = pd.read_csv(DATA_PATH)

y = df.pop(_TARGET)
X = df
```

Helper functions

```
def get_regression_metrics(model, X_test, y_test, X_train, y_train):
    """
    Calculates and prints key regression performance metrics (MAE, MSE, RMSE and R2).

    Parameters:
    model: A fitted sklearn estimator or Pipeline.
    X_test: Test features (DataFrame or Array).
    y_test: True target values.
    X_train: Train features.
    y_train: Train target values.


    Outputs:
    mae: Mean average error
    mse: Mean square error
    rmse: Root mean square error
    r2: R2 score
    tr2: Training R2


    Contribution: Co-developed by Gemini Pro 3.1, mostly me.
    """
    y_pred = model.predict(X_test)

    y_train_pred = model_elastic_net.predict(X_train)
    tr2 = r2_score(y_train, y_train_pred)

    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    return mae, mse, rmse, r2, tr2

def print_regression_metrics(test_mae, test_mse, test_rmse, test_r2, train_r2):
    print(f"\nTraining R2: {train_r2:.4f}")
    print(f"Test R2:     {test_r2:.4f}")
    print(f"Difference:  {train_r2 - test_r2:.4f}")
    print()
    print(f"Other metrics:")
    print(f"MAE: {test_mae:.4f}")
    print(f"MSE: {test_mse:.4f}")
    print(f"RMSE: {test_rmse:.4f}")


# These 2 lines were given by Gemini, they force paralelism in GridSearch
from functools import partial
GridSearchCV = partial(GridSearchCV, n_jobs=-1)
```

1) Train and Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, random_state=_RS, test_size=_TEST_PERCENTAGE,
    stratify=y, # To ensure we have the same percentage of decile scores in test and train data
)
```

2) Linear Models

- We found out about **Quantile** and **Ordinal** regressions while brainstorming with Gemini. They improve the model's ability to handle discrete, ranked scales and outlier-heavy data by focusing on thresholds and medians rather than just the mathematical average.
- The **TransformedTargetRegressor** is a wrapper that transforms the target variable to approximate a normal distribution in cases of skewness. Also sugested by Gemini it is useful in conjunction with the **Quantile** regression model. By stabilizing the variance and **reducing skew**, the transformation ensures the model's 'pinball loss' function is minimized across a more uniform error distribution.
- Finally the usage of Splines to break the ~0.5 barries was also sugested by Gemini, which denoted that the linear model can be more flexible when using them.

  > **Toughts and observations:**
  >
  > **AI Acknowledgements** (Gemini):

- The R2 score for these linear models seems to be capped at ~0.5, which leads us to believe that the dataset is more complex and non purely linear. This is corroborated by the attempt of using **SplineTransform** to improve feature set and improving R2 score, which didn't budge, meaning we maxed out the capabilities of linear models for this feature set.
- While using the **TTR** we chose the np.log and np.exp to treat the right skewness of the target variable.
- TODO: calculate acc, precision and recall with rounded values. After training only.

ElasticNet (Polinomial)

Model pipeline

```
model_elastic_net_pipeline = make_pipeline(
    PolynomialFeatures(degree=1, include_bias=False),
    StandardScaler(), # ElasticNet is very sensitive to feature scale.
    ElasticNet(
        alpha=1.0,
        l1_ratio=0.5,
        max_iter=10000,
        tol=0.01
    ),
)
```

Hyper Parameter Optimization

```
# The GridSearchCV was indicated by an LLM in order to optimize hyper parameters

param_grid = {
    'elasticnet__alpha': [0.01, 0.1, 1.0],
    'elasticnet__l1_ratio': [0.5, 0.9, 0.99],
    'polynomialfeatures__degree': [1, 2]
}

model_elastic_net = GridSearchCV(
    model_elastic_net_pipeline,
    param_grid,
    cv=5,
    scoring='neg_mean_squared_error',
    refit=True # Refit with best model at the end
)
model_elastic_net.fit(X_train, y_train)


print(f"Best Params: {model_elastic_net.best_params_}")
```

```
Best Params: {'elasticnet__alpha': 0.01, 'elasticnet__l1_ratio': 0.5, 'polynomialfeatures__degree': 1}
```

Baseline metrics

```
metrics = get_regression_metrics(model_elastic_net, X_test, y_test, X_train, y_train)
print_regression_metrics(*metrics)
```

```
Training R2: 0.4786
Test R2:     0.4915
Difference:  -0.0129

Other metrics:
MAE: 1.6391
MSE: 4.1000
RMSE: 2.0248
```

Quantile Regression

Model pipeline

```
model_quantile_base_pipeline = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False),
    StandardScaler(),
    QuantileRegressor(
        quantile=0.5,
        alpha=1.0,
        solver='highs'
    ),
)

model_quantile_pipeline = TransformedTargetRegressor(
    regressor=model_quantile_base_pipeline,
    func=np.log,
    inverse_func=np.exp
)
```

Hyper Parameter Optimization

```
param_grid_quantile = {
    'regressor__quantileregressor__quantile': [0.25, 0.5, 0.75],
    'regressor__quantileregressor__alpha': [0.01, 0.1, 1.0],
    'regressor__polynomialfeatures__degree': [1, 2]
}

model_quantile = GridSearchCV(
    model_quantile_pipeline,
    param_grid_quantile,
    cv=5,
    scoring='neg_mean_squared_error',
    refit=True
)
model_quantile.fit(X_train, y_train)

print(f"Best Params: {model_quantile.best_params_}")
```

```
Best Params: {'regressor__polynomialfeatures__degree': 2, 'regressor__quantileregressor__alpha': 0.01, 'regressor__quantileregressor__quantile': 0.5}
```

Metrics

```
metrics_quantile = get_regression_metrics(model_quantile, X_test, y_test, X_train, y_train)
print_regression_metrics(*metrics_quantile)
```

```
Training R2: 0.4786
Test R2:     0.4837
Difference:  -0.0051

Other metrics:
MAE: 1.5582
MSE: 4.1631
RMSE: 2.0404
```

Ordinal Regression

Model pipeline

```
model_ordinal_pipeline = make_pipeline(
    PolynomialFeatures(degree=2, include_bias=False),
    StandardScaler(),
    LogisticAT(alpha=1.0)
)
```

Hyper Parameter Optimization

```
param_grid_ordinal = {
    'logisticat__alpha': [0.01, 0.1, 1.0],
    'polynomialfeatures__degree': [1, 2]
}

model_ordinal = GridSearchCV(
    model_ordinal_pipeline,
    param_grid_ordinal,
    cv=5,
    scoring='neg_mean_squared_error',
    refit=True
)
model_ordinal.fit(X_train, y_train)

print(f"Best Params: {model_ordinal.best_params_}")
```

```
Best Params: {'logisticat__alpha': 1.0, 'polynomialfeatures__degree': 1}
```

Metrics

```
metrics_ordinal = get_regression_metrics(model_ordinal, X_test, y_test, X_train, y_train)
print_regression_metrics(*metrics_ordinal)
```

```
Training R2: 0.4786
Test R2:     0.4751
Difference:  0.0035

Other metrics:
MAE: 1.5181
MSE: 4.2324
RMSE: 2.0573
```

ElasticNet (Spline + TTR)

Model pipeline

```
model_elasticnet_spline_base_pipeline = make_pipeline(
    SplineTransformer(n_knots=5, degree=3),
    StandardScaler(),
    ElasticNet(
        alpha=1.0,
        l1_ratio=0.5,
        max_iter=10000,
        tol=0.01
    ),
)

model_elasticnet_spline_pipeline = TransformedTargetRegressor(
    regressor=model_elasticnet_spline_base_pipeline,
    func=np.log,
    inverse_func=np.exp
)
```

Hyper Parameter Optimization

```
param_grid_elasticnet_spline = {
    'regressor__splinetransformer__n_knots': [5, 8],
    'regressor__elasticnet__alpha': [0.01, 0.1, 1.0],
    'regressor__elasticnet__l1_ratio': [0.5, 0.9, 0.99]
}

model_elasticnet_spline = GridSearchCV(
    model_elasticnet_spline_pipeline,
    param_grid_elasticnet_spline,
    cv=5,
    scoring='neg_mean_squared_error',
    refit=True
)
model_elasticnet_spline.fit(X_train, y_train)

print(f"Best Params: {model_elasticnet_spline.best_params_}")
```

```
Best Params: {'regressor__elasticnet__alpha': 0.01, 'regressor__elasticnet__l1_ratio': 0.5, 'regressor__splinetransformer__n_knots': 5}
```

Metrics

```
metrics_elasticnet_spline = get_regression_metrics(model_elasticnet_spline, X_test, y_test, X_train, y_train)
print_regression_metrics(*metrics_elasticnet_spline)
```

```
Training R2: 0.4786
Test R2:     0.4817
Difference:  -0.0030

Other metrics:
MAE: 1.5402
MSE: 4.1794
RMSE: 2.0444
```

3) Non-Linear Models

> Observations:

- We can't seem to break 0.5 barrier, the literature points out that this is normal for human behaviour variance explainance. Will have to review with professor

Random forest

Model pipeline

```
model_random_forest_pipeline = make_pipeline(
    RandomForestRegressor(
        n_estimators=200,
        max_features='sqrt',    # random subset of features
        random_state=_RS
    ),
)
```

Hyper parameter optimization

```
param_grid_random_forest = {
    'randomforestregressor__n_estimators': [100, 200, 300],
    'randomforestregressor__max_depth': [3, 6, 9],
    'randomforestregressor__min_samples_leaf': [1, 2, 4],
}

model_random_forest = GridSearchCV(
    estimator=model_random_forest_pipeline,
    param_grid=param_grid_random_forest,
    cv=5,
    scoring='neg_mean_absolute_error',
    refit=True
)

model_random_forest.fit(X_train, y_train)

print(f"Best Params: {model_random_forest.best_params_}")
```

```
Best Params: {'randomforestregressor__max_depth': 9, 'randomforestregressor__min_samples_leaf': 1, 'randomforestregressor__n_estimators': 300}
```

Metrics

```
metrics_random_forest = get_regression_metrics(model_random_forest, X_test, y_test, X_train, y_train)
print_regression_metrics(*metrics_random_forest)
```

```
Training R2: 0.4786
Test R2:     0.5034
Difference:  -0.0248

Other metrics:
MAE: 1.6049
MSE: 4.0042
RMSE: 2.0010
```

3) Classification into decile bins

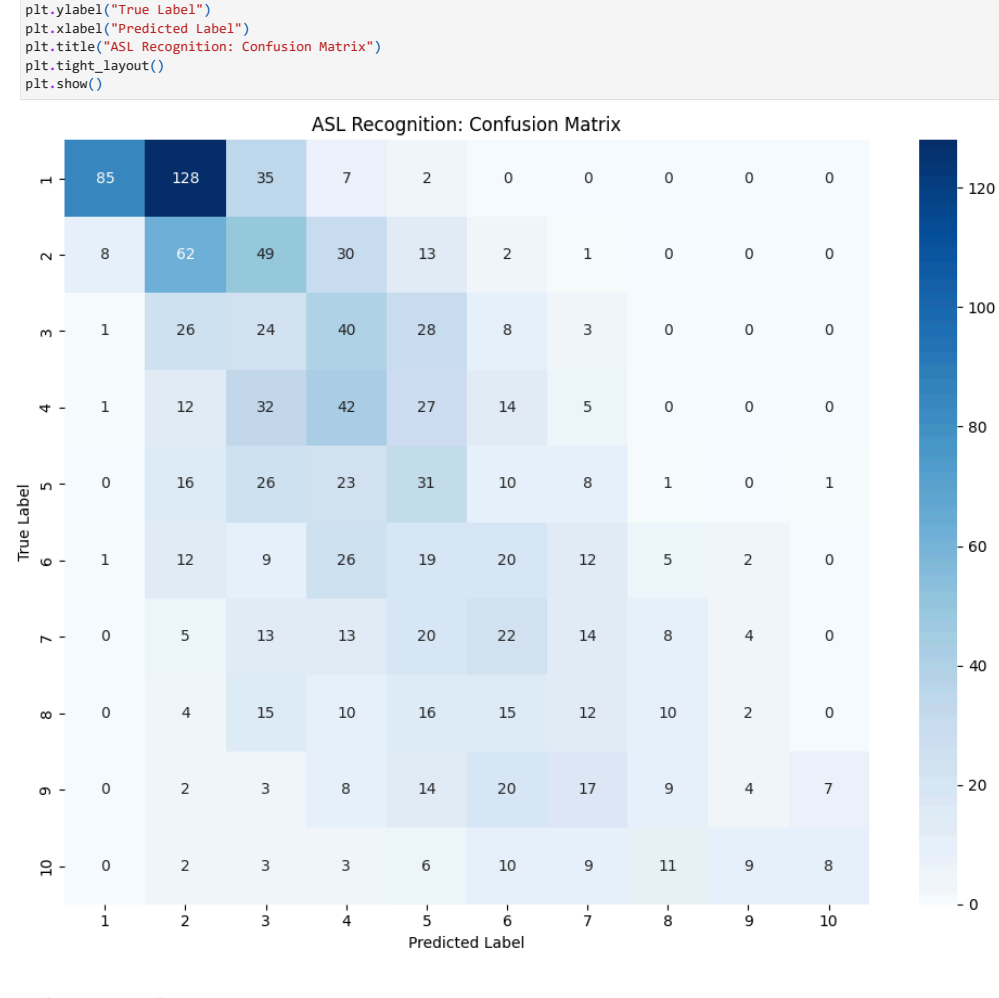
Scoring

```
# Could do top 2 accuracy and metrics, this is, accepting a response beeing + or - one off as it beeing correct.
# it would increase accuracy immediatly and it would help to better visualize this rudimentary classification.

class_model = model_elasticnet_spline

class_y_pred = np.clip(np.round(class_model.best_estimator_.predict(X_test)).astype(int), 1, 10)
class_y_true = y_test.astype(int)

class_precision = precision_score(class_y_true, class_y_pred, average='weighted', zero_division=0)
class_recall    = recall_score(class_y_true, class_y_pred, average='weighted', zero_division=0)
class_f1        = f1_score(class_y_true, class_y_pred, average='weighted', zero_division=0)
class_report    = classification_report(class_y_true, class_y_pred, zero_division=0)

print(f"Precision (weighted): {class_precision:.4f}")
print(f"Recall    (weighted): {class_recall:.4f}")
print(f"F1        (weighted): {class_f1:.4f}")
print()
print(class_report)

# I found out some models have fewer R2 but are closer to the real decile bin (ordinal is the better one at functionning as a classifier)
```

```
Precision (weighted): 0.3472
Recall    (weighted): 0.2429
F1        (weighted): 0.2539

              precision    recall  f1-score   support

           1       0.89      0.33      0.48       257
           2       0.23      0.38      0.29       165
           3       0.11      0.18      0.14       130
           4       0.21      0.32      0.25       133
           5       0.18      0.27      0.21       116
           6       0.17      0.19      0.18       106
           7       0.17      0.14      0.16        99
           8       0.23      0.12      0.16        84
           9       0.19      0.05      0.08        84
          10       0.50      0.13      0.21        61

    accuracy                           0.24      1235
   macro avg       0.29      0.21      0.21      1235
weighted avg       0.35      0.24      0.25      1235
```

Visualization

```
cm = confusion_matrix(y_test, class_y_pred)

plt.figure(figsize=(10, 8))

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=range(1,11), yticklabels=range(1,11))

plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.title("ASL Recognition: Confusion Matrix")
plt.tight_layout()
plt.show()
```



4) Learning Curves

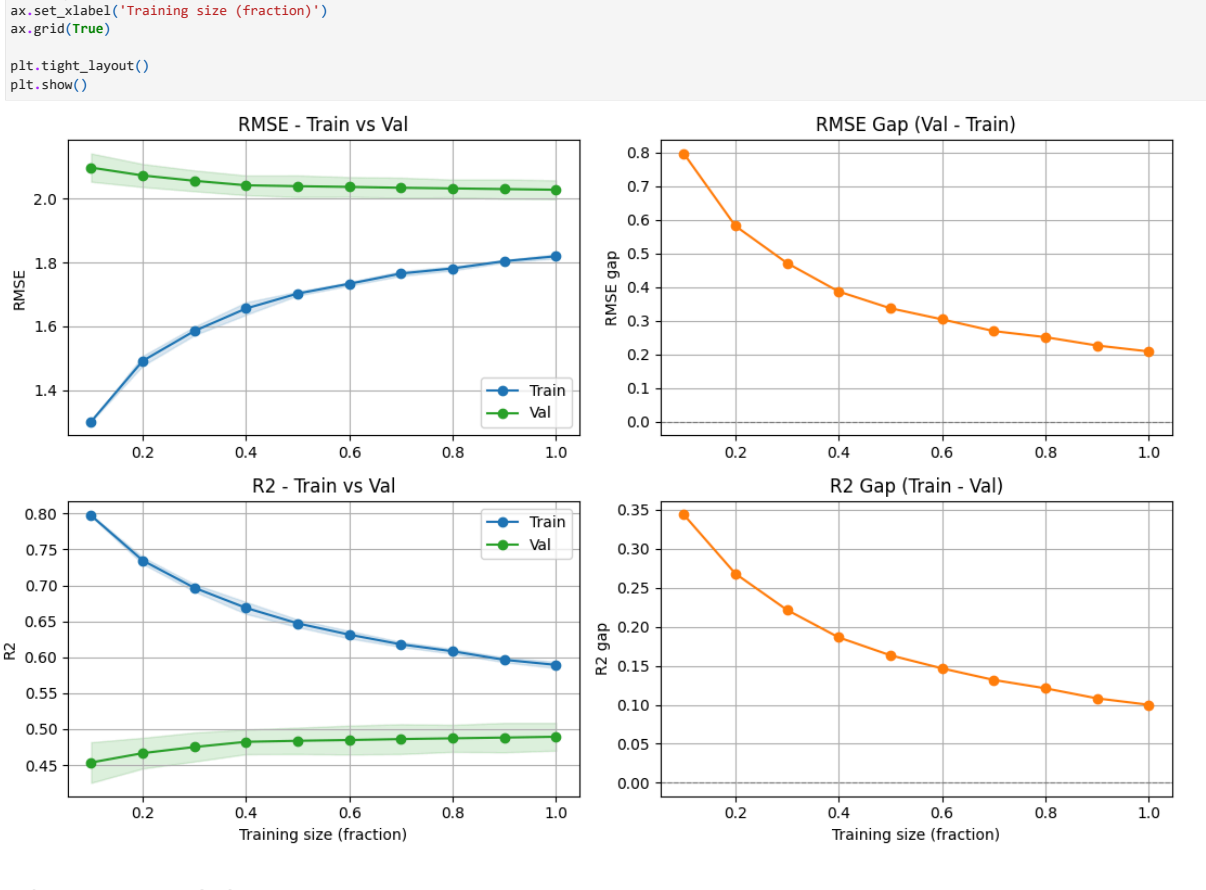
- This section was made with the help of Claude Sonnet 4.6. It helps us understand model fit and overfit trends as the data increases.

  > Thoughs and conclusions:

  > AI Acknowledgement:

- Random forest achived the highest score in R2 but due to a slight overfit.

```
sizes = np.linspace(0.1, 1.0, 10)
lc_model = model_random_forest # Change here to see other models

def _lc(scoring):
    _, tr, vl = learning_curve(
        lc_model.best_estimator_, X, y,
        train_sizes=sizes, cv=5, scoring=scoring, n_jobs=-1
    )
    return tr, vl

tr_mse, vl_mse = _lc('neg_mean_squared_error')
tr_r2,  vl_r2  = _lc('r2')

tr_rmse_m, tr_rmse_s = np.sqrt(-tr_mse).mean(1), np.sqrt(-tr_mse).std(1)
vl_rmse_m, vl_rmse_s = np.sqrt(-vl_mse).mean(1), np.sqrt(-vl_mse).std(1)
tr_r2_m,   tr_r2_s   = tr_r2.mean(1),  tr_r2.std(1)
vl_r2_m,   vl_r2_s   = vl_r2.mean(1),  vl_r2.std(1)

fig, axes = plt.subplots(2, 2, figsize=(11, 7))

# RMSE - Train vs Val
ax = axes[0, 0]
ax.plot(sizes, tr_rmse_m, marker='o', label='Train', color='tab:blue')
ax.plot(sizes, vl_rmse_m, marker='o', label='Val',   color='tab:green')
ax.fill_between(sizes, tr_rmse_m - tr_rmse_s, tr_rmse_m + tr_rmse_s, alpha=0.15, color='tab:blue')
ax.fill_between(sizes, vl_rmse_m - vl_rmse_s, vl_rmse_m + vl_rmse_s, alpha=0.15, color='tab:green')
ax.set_title('RMSE - Train vs Val')
ax.set_ylabel('RMSE')
ax.legend()
ax.grid(True)

# RMSE gap
ax = axes[0, 1]
ax.plot(sizes, vl_rmse_m - tr_rmse_m, marker='o', color='tab:orange')
ax.axhline(0, color='gray', linestyle='--', linewidth=0.8)
ax.set_title('RMSE Gap (Val - Train)')
ax.set_ylabel('RMSE gap')
ax.grid(True)

# R2 — Train vs Val
ax = axes[1, 0]
ax.plot(sizes, tr_r2_m, marker='o', label='Train', color='tab:blue')
ax.plot(sizes, vl_r2_m, marker='o', label='Val',   color='tab:green')
ax.fill_between(sizes, tr_r2_m - tr_r2_s, tr_r2_m + tr_r2_s, alpha=0.15, color='tab:blue')
ax.fill_between(sizes, vl_r2_m - vl_r2_s, vl_r2_m + vl_r2_s, alpha=0.15, color='tab:green')
ax.set_title('R2 - Train vs Val')
ax.set_ylabel('R2')
ax.set_xlabel('Training size (fraction)')
ax.legend()
ax.grid(True)

# R2 gap
ax = axes[1, 1]
ax.plot(sizes, tr_r2_m - vl_r2_m, marker='o', color='tab:orange')
ax.axhline(0, color='gray', linestyle='--', linewidth=0.8)
ax.set_title('R2 Gap (Train - Val)')
ax.set_ylabel('R2 gap')
ax.set_xlabel('Training size (fraction)')
ax.grid(True)

plt.tight_layout()
plt.show()
```



5) Save Models

```
MODELS_DIR = "../models"
os.makedirs(MODELS_DIR, exist_ok=True)

models = {
    "elasticnet_polynomial": model_elastic_net,
    "quantile_ttr":          model_quantile,
    "ordinal":               model_ordinal,
    "elasticnet_spline_ttr": model_elasticnet_spline,
    "random_forest":         model_random_forest,
}

for name, model in models.items():
    path = os.path.join(MODELS_DIR, f"{name}.joblib")
    joblib.dump(model, path)
    print(f"Saved: {path}")
```

```
Saved: ../models/elasticnet_polynomial.joblib
Saved: ../models/quantile_ttr.joblib
Saved: ../models/ordinal.joblib
Saved: ../models/elasticnet_spline_ttr.joblib
Saved: ../models/random_forest.joblib
```

Ethical Considerations

Ethics is a big part of this project. Given the fact that our study aims to bring racial bias to light, it's important that we follow very strict ethical practices.

Ethics in Feature Selection

In terms of features for the training of our first model, we focused purely on the defendants personal features and criminal record. We did not take into account any information that is not a personal characteristic of the defendant.

We also had to take into account that the COMPAS decile score is calculated before a sentence is given. Because of this, we did not take into account any information linked to events that happen after the COMPAS screening where the score was attributed to the subject.

Here are some examples of those decisions:

- **Exclude jail time information**: As mentioned before, the score is calculated before the sentence is given, to help the judge make a better decision. So it is not a cause for the score, instead it is a consequence of it.
- **Not measuring arrest time**: Since we had both the offense date and the arrest date, we could have included the time it took for the subject to get arrested. However, we decided **not to** because that would have more to do with police efficiency than the actual danger of the person.
- **Derive more criminal record information**: We decided to derive a lot of metrics from the original criminal statistics, to test some bias hypothesis. It is easier to prove the bias if correlation to the final score is bigger for the race variable than for the criminal flags like `velocity_repeat_offender`.

Scope of our Ethics

This project aims to expose the racial bias of the COMPAS score. However the score is only calculated when the subject is arrested. There is already a level of bias on the decision to arrest someone, by part of the police. What we are trying to say is that it is hard to mitigate the bias when the existence of the records in itself could happen due to bias.

However this is something we cannot control, and is totally out of our scope. We fully accept the risk that bias may persist simply through the fact that some criminals don't get arrested, and some people do unjustly.